

DeepDash

World Models for Geometry Dash

Florent Tariolle & Maël Planchot

Representation Learning

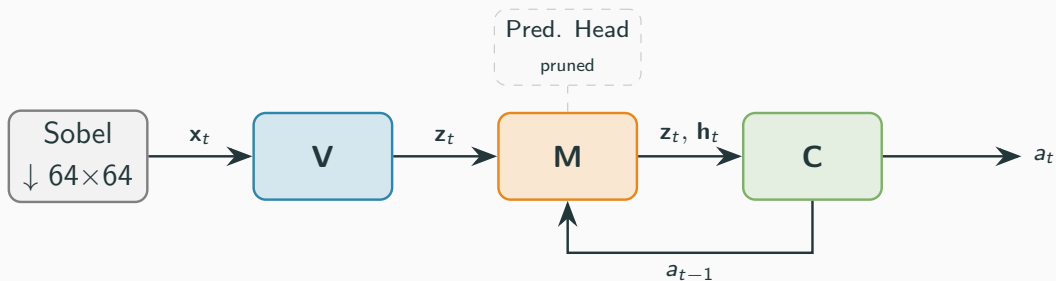
Motivation

Why World Models?

- **Geometry Dash:** deterministic, binary action, precision timing
- **Real-time:** 30 FPS $\rightarrow$ reaction in $<33\text{ms}$
- **Model-free RL:** needs millions of game interactions $\rightarrow$ infeasible
- **World Models:** learn the environment, train the policy *entirely in imagination*
- **Goal:** master unseen levels with a tiny real-game footprint

Architecture Overview

Inference Pipeline



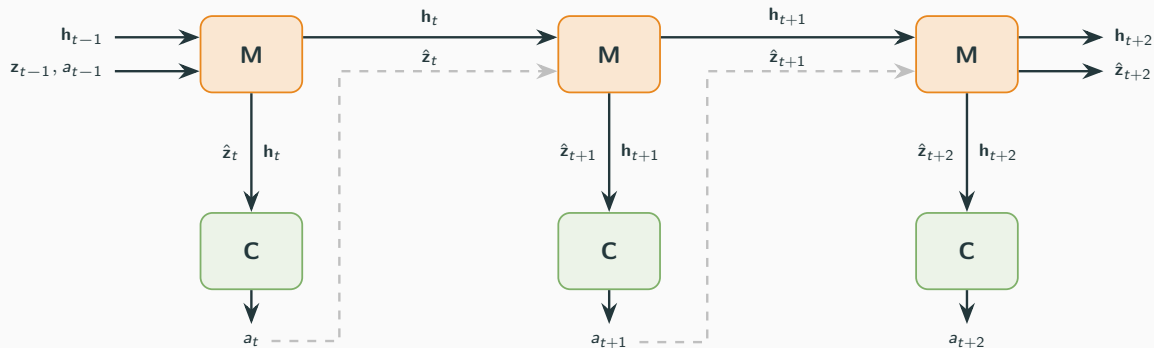
$x_t \in \mathbb{R}^{64 \times 64}$
Sobel frame

$z_t \in \{0, \dots, 999\}^{64}$
discrete tokens

$h_t \in \mathbb{R}^{384}$
hidden state

$a_t \in \{0, 1\}$
jump/idle

Dream Rollout



- Seeded with $K=4$ real frames, then dreams autoregressively
- Rollout stops when death predicted or max steps reached

Data Collection

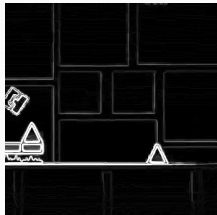
Data Collection



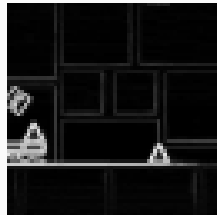
Raw 1080p



Square crop



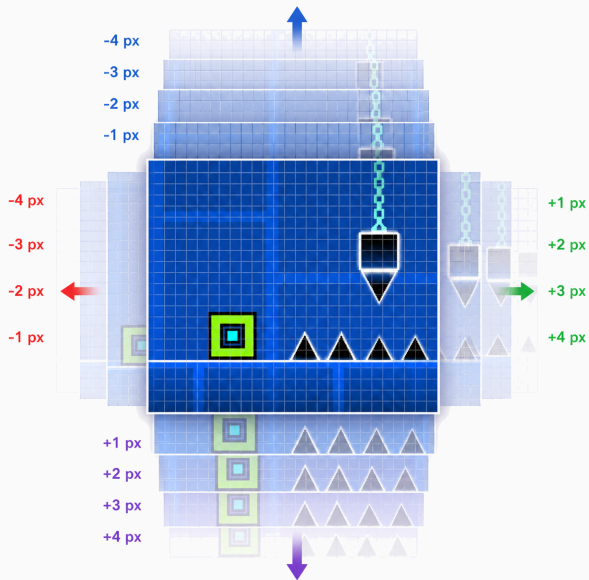
Sobel edges



64×64

- Live gameplay at **30 FPS**, death detection via process memory reading
- Frames + actions recorded simultaneously, saved as `.npy` per episode
- **Death episodes**: $\sim 4,250$ episodes, $\sim 220\text{K}$ frames (intentional deaths)
- **Expert episodes**: 35 clean runs, $\sim 30\text{K}$ frames (for BC + rebalancing)

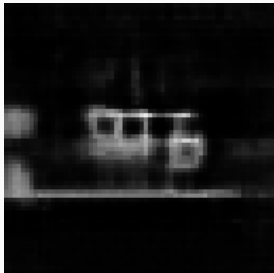
Data Augmentation: Spatial Shifts



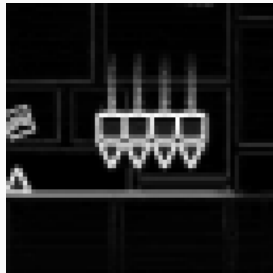
Phase 1: Vision

Vision Model: β -VAE $\rightarrow$ FSQ-VAE

- β -VAE: Gaussian posterior. No spatial structure.
- VQ-VAE: discrete, spatially localized tokens $\Rightarrow$ sharp. But codebook collapse, commitment cost tuning.
- FSQ-VAE (Mentzer et al., 2023): round to L fixed levels. No codebook collapse, no auxiliary losses, 100% utilization by construction.

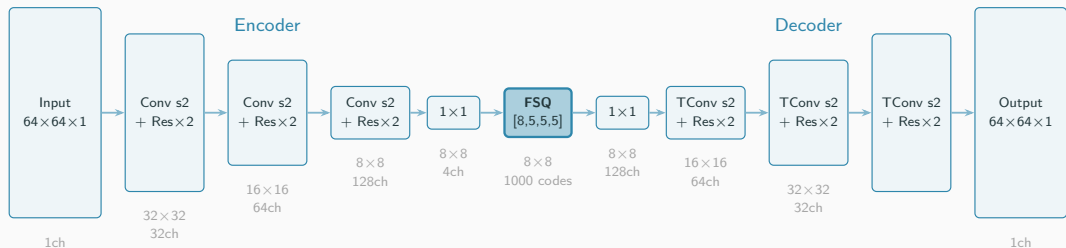


β -VAE: blurry



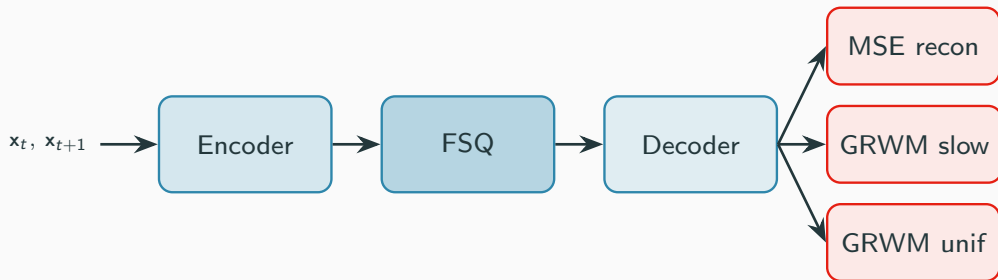
FSQ-VAE: sharp

FSQ-VAE: Architecture



Levels [8, 5, 5, 5] $\Rightarrow$ 1,000 codes
 8×8 grid = **64 tokens/frame**

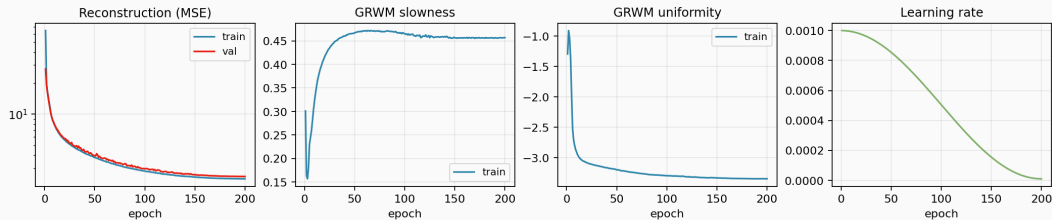
FSQ-VAE: Training Losses



- **MSE** reconstruction on $(\mathbf{x}_t, \mathbf{x}_{t+1})$ pairs (STE through quantizer)
- **GRWM slowness**: penalizes large $\|\mathbf{z}_e^{t+1} - \mathbf{z}_e^t\|^2$ – temporally smooth latents
- **GRWM uniformity**: spreads $\mathbf{z}_e$ across the FSQ hypercube

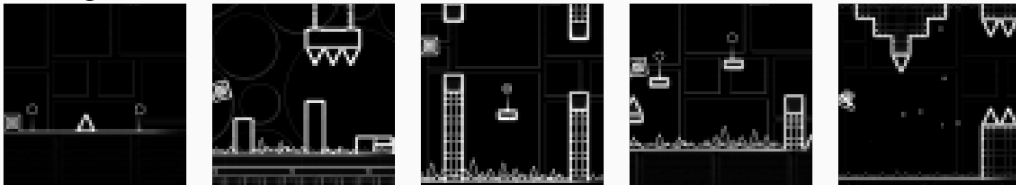
FSQ-VAE: Training

- AdamW, batch 2048, $\sim 252\text{K}$ frame pairs
- GRWM weights: $\alpha_{\text{slow}} = 0.1$, $\alpha_{\text{unif}} = 0.01$

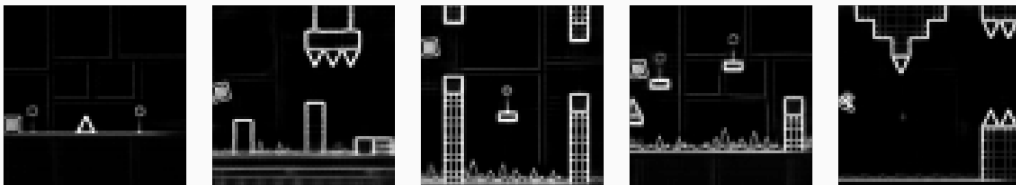


FSQ-VAE: Reconstructions

Original



Reconstruction



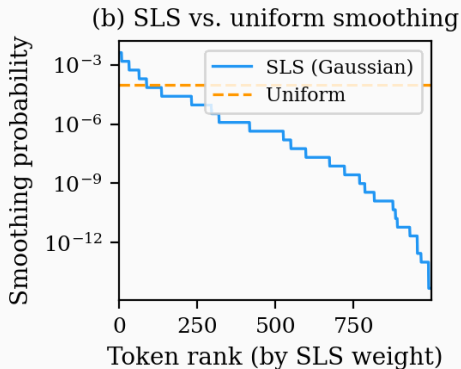
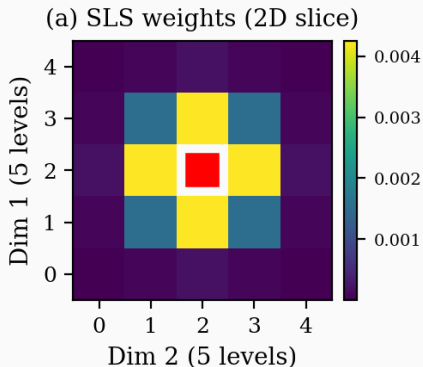
51× compression: 32,768 → 638 bits/frame

Phase 2: Memory

Structured Label Smoothing (SLS)

Problem: CE penalises a near-miss neighbour as harshly as a random code.

SLS: one-hot $\rightarrow$ Gaussian over FSQ coordinates.

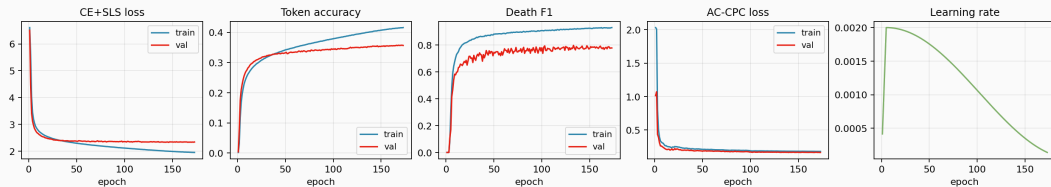


(a) Gaussian decay over lattice distance.

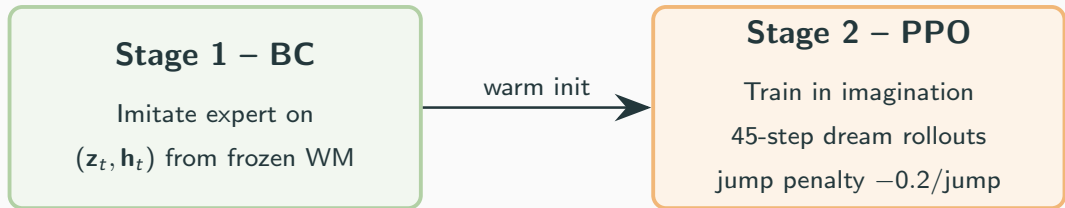
(b) SLS concentrates mass on near-misses, vs. uniform's flat smear.

Transformer: Training

- AdamW, batch 512, dropout 0.1
- Death oversampling $5\times$; token noise: 5% random + 5% FSQ-neighbour

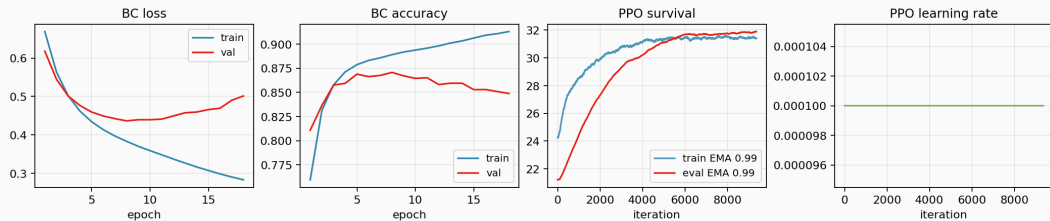


Phase 3: Controller



Zero-shot deployment on the real game after PPO.

Controller: Training Curves



Pipeline Timing

Pipeline Timing

Mean per-stage inference time on RTX 2060 ($n \approx 5K$ frames).

Stage	Mean (ms)
Screen capture	2.4
Crop	0.5
Grayscale	0.5
Sobel	2.6
Downscale	2.1
FSQ encode	1.4
Transformer	4.2
Controller	1.1
Total	15.0

Limitations & Future Work

Limitations & Future Work

- **Sample efficiency:** trained on only 2h20 of gameplay – world models make this tractable
- **Dream horizon:** dreams stay coherent for tens to hundreds of frames, then hallucinate – repeated obstacles, impassable sections, or empty levels
- **Scope:** trained on simple mechanics (portals, jump pads, orbs, standard gravity). Complex levels with noisy backgrounds and harder dynamics are the next step.
- **Inelegant pipeline:** too many losses, encoder still trained with a pixel reconstruction objective, sequential trainings with frozen stages. JEPA is the principled fix – our adaptation did not outperform V3, but we cannot tell whether we mis-adapted it or whether discrete offline setups are less amenable to it than continuous ones.

Demo



youtube.com/watch?v=LkUlyeEwPEs

References

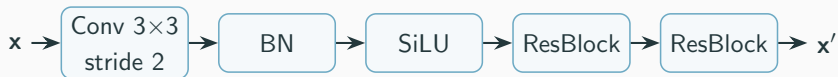
- Micheli, V. et al. (2023). *Transformers are Sample-Efficient World Models (IRIS)*. ICLR
- Mentzer, F. et al. (2024). *Finite Scalar Quantization: VQ-VAE Made Simple*. ICLR
- Burchi, M. & Timofte, R. (2025). *TWISTER: Transformer-based World Models with AC-CPC*. ICLR
- Su, J. et al. (2024). *RoFormer: Enhanced Transformer with Rotary Position Embedding*. Neurocomputing
- Schulman, J. et al. (2017). *Proximal Policy Optimization Algorithms*. Preprint
- Ha, D. & Schmidhuber, J. (2018). *World Models*. NeurIPS
- Hafner, D. et al. (2025). *Mastering Diverse Control Tasks through World Models (DreamerV3)*. Nature

- Vali, M. et al. (2026). *iFSQ: Improving FSQ for Image Generation*. ICLR
- Lin, T.-Y. et al. (2017). *Focal Loss for Dense Object Detection*. ICCV
- Maes, L. et al. (2026). *LeWorldModel: Stable End-to-End JEPA from Pixels*. Preprint
- Xia, Z. et al. (2025). *Clone Deterministic 3D Worlds with Geometrically-Regularized World Models (GRWM)*. Preprint

Appendix

FSQ-VAE: Inside an Encoder Block

Each encoder level:



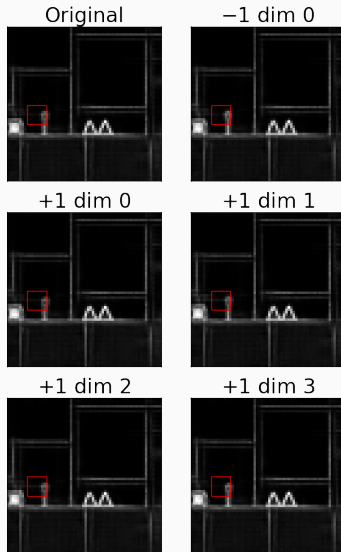
Each **ResBlock**:



Training: FSQ-Neighbor Token Noise

5% per context token: replace by a ± 1 neighbour in one FSQ dimension.

- Visually near-identical
- Treat near-miss codes as equivalent
- Mirrors SLS on the input side



Tried After V3 – Did Not Outperform Yet

More elegant, faster to train, but less stable / less precise on the real game:

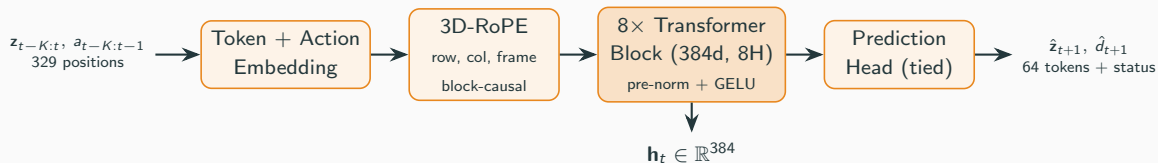
- **End-to-end V+M training** (JEPA way) – V and M trained jointly with only *two* losses (MSE on pixels + CE on tokens). Stable, but the resulting FSQ is harder to act on than V3's separately-trained FSQ.
- **AdaLN-Zero action conditioning**: actions removed from the sequence, injected via AdaLN. Best CPC ever, but 5 runs diverged before QK-norm and gate-bias-1 init unblocked it.
- **MLPPolicy on $\mathbf{h}_t$ only**: $2\times$ faster convergence, smallest train/eval gap, but the V3 CNN still wins on real-game with the V3 FSQ.

Net: V3 stays the best deployment despite V4–V6 being more elegant on paper.

What Didn't Work (or Didn't Help)

- β -VAE: blurry, large grid search $\rightarrow$ abandoned
- MaskGIT: degraded on 64 tokens, parallel single-step better
- Soft continuation gating: agent gamed airtime
- PMPO: oscillated with binary action space
- Scaling to 512d: bigger, slower, no real-game gain on the new latents.

Transformer: Architecture

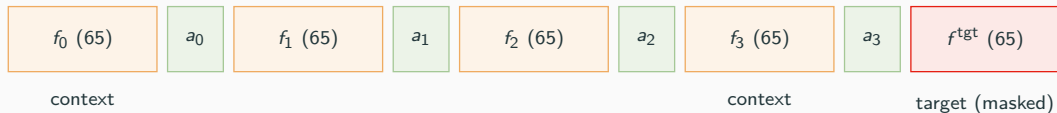


Sequence: $[f_0 a_0 f_1 a_1 \dots f_{K-1} a_{K-1} f^{\text{tgt}}]$, target = learnable mask embed

Vocab: $|\mathcal{V}| = 1,002$ (1000 visual + ALIVE + DEATH) 14.7M params

Transformer: Sequence Layout

Each frame block has **65 positions**: 64 visual tokens + 1 status token (ALIVE / DEATH). Action tokens are interleaved between frame blocks. The target frame is replaced by a learnable mask embedding – no ground-truth leakage.



- **Block-causal** attention: bidirectional within each block, causal across.
- **3D-RoPE**: head dim split into row / col / frame bands, lower θ for spatial axes.
- **Parallel decode**: all 65 target positions predicted in one forward pass.

Transformer: Auxiliary Contrastive Loss (AC-CPC)

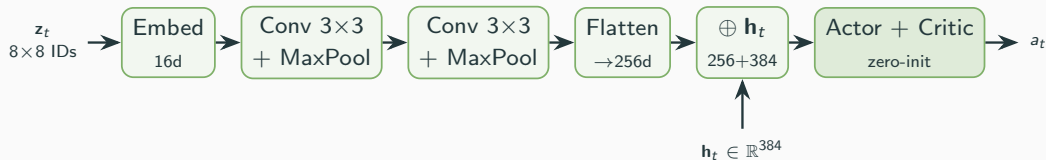
Beyond next-frame CE, the transformer is trained with an **action-conditional contrastive predictive coding** loss (TWISTER, Burchi & Timofte 2025):

- For each $k \in \{1, \dots, K\}$, predict $\mathbf{h}_{t+k}$ from $\mathbf{h}_t + a_{t:t+k-1}$ via an MLP head.
- InfoNCE loss with in-batch negatives (temperature 0.1).
- Forces $\mathbf{h}_t$ to encode action-conditional dynamics, not just static appearance.

Combined loss: $\mathcal{L} = \mathcal{L}_{\text{CE+SLS}} + 0.1 \cdot \mathcal{L}_{\text{AC-CPC}}$.

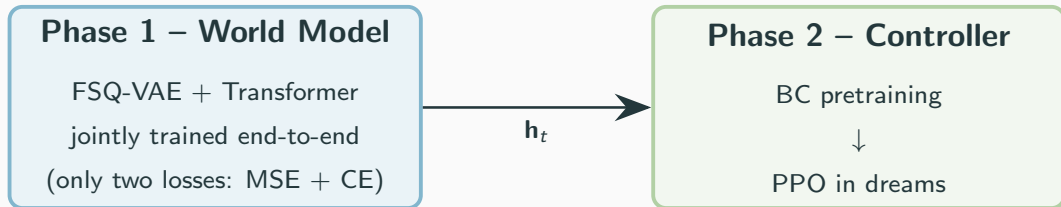
Controller: Architecture

CNNPolicy ($\sim 45\text{K}$ params): IRIS/DIAMOND-style CNN on the 8×8 token grid, fused with $\mathbf{h}_t$.



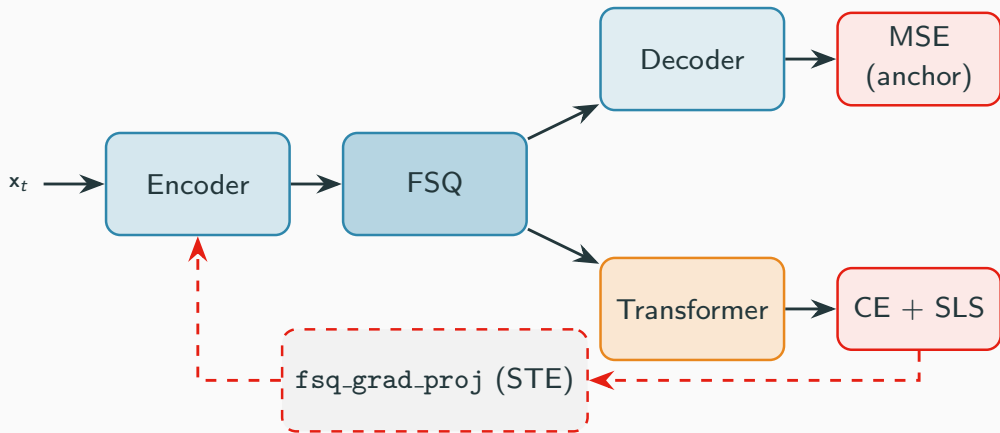
Auxiliary head: **multi-token prediction** – predict next 8 actions (timing prior).

JEPA Way – Two Training Phases



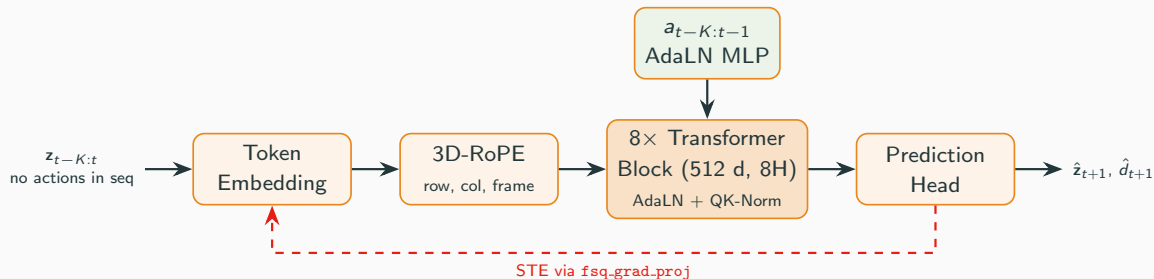
V and M become inseparable: prediction gradients flow back into the encoder via STE.
Elegant, faster, but the resulting FSQ is harder to act on than V3's.

JEPA Way – V + M: Joint Training



Pure-JEPA (no MSE anchor) suffers codebook collapse – pixel anchor is the regularizer.

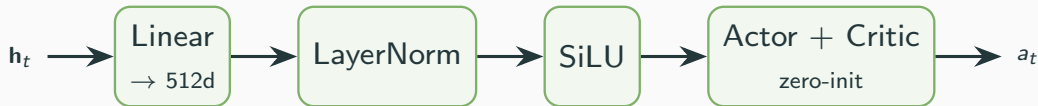
Transformer with AdaLN-Zero (V4–V6)



- Actions leave the sequence (injected via AdaLN) and the target block is fused into the last context position. Sequence: 329 $\rightarrow$ 260 (−4 actions, −65 target).
- Required **QK-norm** and **gate-bias-1 init** – otherwise 5/5 runs diverged.
- Best CPC ever (0.13), but no real-game gain over V3.

MLPPolicy on $\mathbf{h}_t$ (V4–V6)

MLPPolicy ($\sim 265\text{K}$ params): 1-layer MLP actor-critic on $\mathbf{h}_t$ alone, no token grid.



- 2–5 $\times$ faster PPO convergence than V3's CNNPolicy.
- Smallest train/eval gap of any controller we trained.
- Beats CNN on the V4–V6 latents – but on V3 latents the CNN still wins on real game.